

CampusFind

A Cloud-Based Campus Lost & Found System

CSBC 252: Introduction to Cloud Computing

Semester Group Project / Capstone Project Pack

Prepared for: AWS Free Tier Cloud Application Project

Project type: Simple 3-day build suitable for a student group

Core AWS services: EC2, RDS, S3, IAM, Security Groups, CloudWatch

This document contains the project proposal, system design guide, implementation plan, AWS deployment guide, testing guide, documentation checklist, and video presentation outline.

Table of Contents

1. Project Proposal
2. Project Guide
3. System Design Document
4. AWS Architecture and Deployment Guide
5. Database Design
6. Security Plan
7. Testing and Validation Plan
8. Deliverables Checklist
9. 3-Day Work Plan
10. Video Presentation Script Outline
11. Final Technical Report Template

1. Project Proposal

Project Title: CampusFind: A Cloud-Based Campus Lost & Found System

Problem Statement

Students often lose important items such as ID cards, keys, chargers, books, phones, bags, and documents on campus. In many schools, lost and found reports are handled through WhatsApp groups, word of mouth, notice boards, or informal announcements. This makes it difficult to search, verify ownership, and track whether an item has been returned. CampusFind provides a centralized cloud-based platform where users can post lost or found items and communicate clear contact details.

Objectives

- Allow users to register, log in, and manage their own lost/found item posts.
- Provide CRUD operations for item records: create, read, update, and delete.
- Store uploaded item images securely in Amazon S3 instead of EC2 local storage.
- Store user and item records in Amazon RDS.
- Deploy the application publicly on Amazon EC2 using AWS Free Tier.
- Use IAM, Security Groups, environment variables, and CloudWatch to demonstrate cloud security and monitoring.

Target Users

- Students who lost items on campus.
- Students or staff who found items and want to report them.
- Campus administrators or SRC/IT staff who manage lost and found records.

Proposed Solution

CampusFind will be a responsive web application that lets authenticated users post lost or found items with title, category, location, description, contact information, status, and an optional image.

Administrators can review posts, remove inappropriate records, and mark items as claimed or resolved.

Recommended Technology Stack

- Backend and frontend: Django with Bootstrap templates.
- Database: Amazon RDS PostgreSQL or MySQL.
- File storage: Amazon S3.
- Hosting: Amazon EC2 Ubuntu instance.
- Monitoring: Amazon CloudWatch.
- Version control: GitHub.

2. Project Guide

CampusFind should be built as a simple but complete CRUD application. Keep the scope small so the group can complete it in 3 days while still meeting the cloud project requirements.

Core Features

- Home page with project description and navigation.
- User registration, login, and logout.
- Add lost/found item form.
- View all listed items in a card or table layout.
- Search/filter by title, category, status, or location.
- View item detail page.
- Edit/delete only your own posts.
- Upload item image to Amazon S3.
- Admin dashboard through Django Admin.

Keep Out of Scope

- No chat system.
- No payment system.
- No complex notification system.
- No mobile app.
- No advanced analytics beyond simple counts.

Minimum Pages

- 1 Home page
- 2 Register page
- 3 Login page
- 4 All items page
- 5 Add item page
- 6 Item detail page
- 7 My posts dashboard
- 8 Django admin panel

3. System Design Document

Use Cases

- A user creates an account.
- A user logs in.
- A user posts a lost item.
- A user posts a found item.
- A user uploads an item image.

- A user searches for items.
- A user edits or deletes their own post.
- An admin reviews and manages all posts.
- An admin marks an item as claimed/resolved.

Simple Architecture Diagram Description

Internet users access the CampusFind web application through a public EC2 instance. The Django application running on EC2 connects to Amazon RDS for structured data and sends uploaded item images to Amazon S3. IAM controls access permissions. Security Groups restrict network traffic. CloudWatch collects basic logs and EC2 metrics.

Layer	Component	Purpose
Presentation	Django templates + Bootstrap	Responsive user interface
Application	Django app on EC2	Business logic, authentication, CRUD
Database	Amazon RDS	Stores users and item records
Storage	Amazon S3	Stores uploaded item images
Security	IAM + Security Groups	Least privilege and network control
Monitoring	CloudWatch	Metrics and log monitoring

4. AWS Architecture and Deployment Guide

AWS Services Required

- IAM: Create least-privilege access for S3 and EC2 operations.
- EC2: Host the Django web application.
- Security Groups: Allow HTTP/HTTPS; restrict SSH to trusted IP only; allow RDS only from EC2.
- RDS: Store user accounts and item records.
- S3: Store uploaded images.
- CloudWatch: Monitor EC2 metrics and application logs.
- Application Load Balancer: Optional bonus only.

Deployment Steps

- 1 Create a GitHub repository and push the project code.
- 2 Launch an Ubuntu EC2 instance under Free Tier.
- 3 Configure Security Group: allow port 22 for SSH from your IP, port 80 for HTTP, and optionally 443 for HTTPS.
- 4 Create an RDS MySQL/PostgreSQL database and allow inbound access only from the EC2 security group.
- 5 Create an S3 bucket for item images. Block public access unless using controlled object access; store only the object URL/path needed by the app.
- 6 Create IAM policy/role or access keys with minimum S3 permissions needed by the app.
- 7 SSH into EC2, install Python, pip, venv, Git, Nginx/Gunicorn if needed.
- 8 Clone the repository, configure .env variables, install requirements, run migrations, collect static files, and start the application.
- 9 Confirm the app is reachable publicly through the EC2 public IP or domain.
- 10 Open CloudWatch and capture EC2 CPU/network metrics and logs for the proof portfolio.

5. Database Design

Users Table

- id
- username/name
- email
- password hash
- role: user/admin
- created_at

Items Table

- id
- user_id
- title

- description
- category
- status: Lost, Found, Claimed
- location
- contact
- image_url
- created_at
- updated_at

Entity Relationship

One user can create many item posts. Each item post belongs to one user. Admin users can manage all posts through the admin panel.

6. Security Plan

- Use Django password hashing and built-in authentication.
- Do not store AWS keys, database passwords, or secret keys directly in the source code. Use environment variables.
- Configure Security Groups using least privilege.
- Restrict database access so only the EC2 instance can connect to RDS.
- Do not store uploaded images on EC2 local storage; send uploads to S3.
- Give the application only the S3 permissions it needs, such as put/get object for the specific bucket.
- Validate uploaded file types and limit upload size.
- Use HTTPS if time allows.
- Keep DEBUG=False in production.

7. Testing and Validation Plan

Functional Tests

- Register a new user.
- Log in and log out.
- Create a lost item post.
- Create a found item post.
- Upload an image and confirm it appears in S3.
- View item list and item detail.
- Edit own post.
- Delete own post.
- Admin marks item as claimed.

Cloud Validation Screenshots

- EC2 instance running.
- RDS database available.
- S3 bucket showing uploaded image objects.
- IAM role or policy used for the project.
- Security Group inbound rules.
- CloudWatch EC2 metrics.
- Public web app URL working in the browser.
- GitHub repository page.

8. Deliverables Checklist

- Project Proposal: title, roles, problem statement, objectives, solution, technologies.
- System Design Document: use cases, ERD, AWS architecture diagram, UI wireframes.

- Deployment Proof Portfolio: screenshots of AWS resources, public app, GitHub URL, CloudWatch metrics.
- Final Technical Report: development lifecycle, architecture, implementation, testing, challenges, future work.
- Pre-recorded Video Presentation: 15–20 minutes with each member explaining personal contribution.

9. 3-Day Work Plan

Day	Main Goal	Tasks
Day 1	Build core app	Set up Django; create models; authentication; item CRUD; Bootstrap pages; local testing.
Day 2	Add AWS services	Create EC2, RDS, S3, IAM; connect app to RDS; configure S3 upload; deploy on EC2.
Day 3	Polish and document	Improve UI; test all features; capture screenshots; prepare proposal/report/portfolio; record

10. Video Presentation Script Outline

- 1 Introduction: project title, problem, and target users.
- 2 Demo: show registration, login, add item, upload image, item list, edit/delete, admin status change.
- 3 Architecture: explain EC2, RDS, S3, IAM, Security Groups, and CloudWatch.
- 4 Database: explain Users and Items tables.
- 5 Security: explain least privilege, environment variables, file upload handling, and restricted RDS access.
- 6 Testing: show proof of successful CRUD, S3 upload, and deployed public access.
- 7 Individual Contributions: each member explains what they built.
- 8 Conclusion: challenges, lessons learned, and future improvements.

11. Final Technical Report Template

- Cover Page
- Abstract / Executive Summary
- Introduction
- Problem Statement
- Objectives
- Scope and Limitations
- System Requirements
- System Design: Use Case, ERD, AWS Architecture, Wireframes
- Implementation Details
- AWS Deployment Process
- Security Configuration
- Testing and Validation
- Challenges Encountered
- Future Enhancements
- Conclusion
- References
- Appendix: Screenshots and GitHub URL

Suggested Group Roles

- Frontend Developer: Bootstrap templates, navigation, item pages.

- Backend Developer: Django models, views, forms, authentication, CRUD.
- Cloud Architect: EC2, Security Groups, architecture diagram.
- Database Lead: RDS setup, ERD, migrations, database validation.
- Storage/Security Lead: S3 upload, IAM, environment variables, security notes.
- Documentation/QA Lead: testing, screenshots, final report, presentation editing.

Future Enhancements

- Email notification when a matching item is posted.
- QR-code tags for departments or notice boards.
- Advanced search and category filters.
- Admin analytics dashboard.
- HTTPS with a domain name.
- Application Load Balancer and multiple EC2 instances for bonus marks.